

3D Graphics Engine & Digital Twin Specification

Source: docs/THREE_D_ENGINE.md • Status: Verified & Synchronized

Product: Cubyntra
Organization: Necookie Labs
Status: Implemented & Tested
Module: src/three/ and src/components/cube/

1. Overview & Architectural Goals

The Cubyntra 3D engine provides an interactive, mathematically faithful digital twin of the physical Rubik's Cube. It executes in client-side WebGL via Three.js.

Core Engineering Invariants

- Visual Follows Model: Three.js is strictly a rendering projection. The logical cube state in src/cube/ is the single source of truth.
- Zero Floating-Point Accumulation Drift: Repeated matrix multiplications during continuous face turns inevitably degrade floating-point orthogonality without rigorous snapping. All layer rotations round cubie coordinates to integer space $\{-1, 0, 1\}$ upon animation completion.
- Smooth 60 FPS Interpolation: Smooth easing transitions between move states.
- Context Loss Resilience: Graceful handling of WebGL device loss and dynamic canvas resize.

2. 27-Cubie Spatial Decomposition

A standard 3 x 3 x 3 Rubik's cube consists of 26 physical pieces plus 1 core mechanism, indexed in discrete spatial coordinates:

$x, y, z \in \{-1, 0, 1\}$

Layers along axes:
X-Axis: Left ($x = -1$), Middle ($x = 0$), Right ($x = +1$)
Y-Axis: Down ($y = -1$), Middle ($y = 0$), Up ($y = +1$)
Z-Axis: Back ($z = -1$), Middle ($z = 0$), Front ($z = +1$)

Cubie Classification:

- Corners (8 pieces): $|x| + |y| + |z| = 3$. Have 3 colored faces, 3 internal faces.
- Edges (12 pieces): $|x| + |y| + |z| = 2$. Have 2 colored faces, 4 internal faces.
- Centers (6 pieces): $|x| + |y| + |z| = 1$. Have 1 colored face, 5 internal faces.
- Core (1 piece): $|x| + |y| + |z| = 0$. Hidden internal pivot mechanism.

Speedcube PBR Mesh Construction & Shared Buffer Geometries

Rather than rudimentary BoxGeometry cubes, Cubyntra constructs cubies using precision-extruded competition speedcube components:

- Chamfered ABS Body: Extruded rounded shape with bevel (createRoundedBoxGeometry(0.985, 0.11, 5)) creating smooth aerodynamic edges. Shaded with matte black ABS plastic MeshPhysicalMaterial (#15161b, roughness 0.55, clearcoat 0.25).
- 3D Beveled Facelet Tiles: Tactile 3D tiles (createRoundedTileGeometry(0.84, 0.13, 0.018)) with quadratic corner fillets and bevel. Mounted onto external faces with slight normal offset, eliminating z-fighting and mimicking physical speedcube tiles.

- PBR Clearcoat Materials: High-gloss specular coating with MeshPhysicalMaterial (roughness 0.32, clearcoat 0.70, clearcoatRoughness 0.18, envMapIntensity 0.90) and authentic competition speedcube color palette:
- White (#f4f5f0), Yellow (#ffd500), Green (#00b35c), Blue (#0a5c9f), Red (#e8132e), Orange (#ff7a00).
- Buffer Geometry Sharing: Shared module-level geometry singletons (getSharedBodyGeometry and getSharedTileGeometry) reuse vertex and index buffers across all 27 cubies, guaranteeing minimal GPU memory overhead and zero garbage collection spikes.

3. Dynamic Layer Rotation & Zero-Drift Snapping

3.1 Grouping and Detaching Mechanics

Directly rotating individual cubies along world axes creates non-affine shearing when rotations are compounded. Cubyntra solves this via temporary dynamic grouping:

- Selection: Identify the 9 cubies whose current rounded world coordinate matches the target slice:
- For U: all cubies where $\text{round}(y) = +1$.
- For R: all cubies where $\text{round}(x) = +1$.
- For F: all cubies where $\text{round}(z) = +1$.
- Parenting: An ephemeral THREE.Group is instantiated at (0, 0, 0). Each of the 9 cubies is reparented to this pivot group using `pivotGroup.attach(cubieMesh)`. Three.js preserves their global world transforms.
- Interpolated Turn: The pivot group's local Euler angle or quaternion is animated over duration T using easing curves:
 $\theta(t) = \theta_{\text{target}} \cdot \text{ease}(t / T)$
- Detaching: Upon reaching θ_{target} , each cubie is reparented back to the primary root scene using `scene.attach(cubieMesh)`.
- Orthonormal Snapping:
 - The world position (x, y, z) is rounded to the nearest integer grid point:
 $x_{\text{clean}} = \text{round}(x), \quad y_{\text{clean}} = \text{round}(y), \quad z_{\text{clean}} = \text{round}(z)$
 - The orientation quaternion q is decomposed into principal orthogonal axes and clamped to exact multiples of $\pi / 2$ radians (90°).

4. Directional 3D Move Arrows

To guide users through solving moves physically, Cubyntra renders animated 3D directional arrows projecting above the active rotating face:



- Geometry: Parametric arc curve (THREE.QuadraticBezierCurve3) extruded with THREE.TubeGeometry, terminating in a conical arrow tip (THREE.ConeGeometry).
- Dynamic Placement: Projected at a normal offset of +0.8 units from the center of the active face.
- Rotation Sense: The arrow arc curve automatically flips based on clockwise (U), counter-clockwise (U'), or double-turn (U2) move specifications.
- Pulsing Animation: Subtle emissive shader pulse synchronizing with playback tempo.

5. Camera & OrbitControls

- Perspective Camera: 45° Field of View (FOV) positioned at (3.6, 3.2, 5.0) looking at (0, 0, 0) providing an isometric three-face perspective (U, F, R).
- OrbitControls Constraints:
 - Damping factor: 0.05 for smooth tactile inertial rotation.
 - Zoom bounds: Minimum distance 3.5, maximum distance 12.0.
 - Pan disabled to ensure the cube remains strictly centered in the viewport.
- Responsive Resize: Automatically recalculates aspect ratio and projection matrix on window/container resize without distortion.

5.1 Studio Environment & Contact Shadow

- PMREM Studio Reflections: Real-time environment reflections generated via THREE.PMREMGGenerator using synthetic

studio light panels (overhead softbox, left/right rim strips, and front fill), casting natural glossy highlights across beveled tile edges.

- Contact Shadow Plane: Projected radial gradient texture positioned at $y = -2.35$ with dynamic scale and opacity breathing in sync with subtle floating idle bobbing.
 - Color Science & Tone Mapping: Configured with `THREE.SRGBColorSpace` and `THREE.ACESFilmicToneMapping` (exposure 1.05) for realistic physical rendering.
-

6. Rendering Performance & Resource Lifecycle

- Disposal: Thorough resource cleanup hooks disposing geometries, materials, and textures when the component unmounts to prevent memory leaks in Single Page Applications.
- Frame Rate Throttling: Renders only when dirty (during active animation or user interaction) to minimize battery consumption on mobile devices.